

Содержание

Библиотека встроенных функций Рефала-5J	2
Функции арифметики	2
Функции Add, Div, Mod, Mul, Sub	3
Функция Divmod	3
Функция Compare	4
Функция Numb	4
Функция Symb	4
Функции запроса и преобразования типов	4
Функция Type	4
Функция Implode	6
Функция Implode_Ext	6
Функции Explode и Explode_Ext	6
Функции Chr, Lower, Ord, Upper	6
Функции работы со стеками	7
Функция Br	7
Функция Cp	7
Функция Dg	7
Функция Rp	8
Функции ввода-вывода	8
Консольный ввод-вывод, функции Card, Print, Prout	8
Дескрипторы файлов, функции Open и Close	8
Чтение и запись файлов, функции Get, Put, Putout, Write	10
Функция ExistFile	10
Функция RemoveFile	10
Взаимодействие с операционной системой	11
Функция Arg	11
Функция GetEnv	11
Функция Exit	11
Функция System	12
Функция Time	12
Функции работы с выражениями	12
Функция Lenw	12
Функции First и Last	12
Запросы к среде исполнения	12
Функция ListOfBuiltin	12
Функция Step	13
Функция TimeElapsed	13
Функция SizeOf	13
Метафункции	14
Функции Mu и Residue	14
Генерация случайных чисел	15

Функция RandomDigit	15
Функция Random	15
Системные функции, сериализация выражений	15
Функция Sysfun	16
Функция DeSysfun	17
Собственные функции данной реализации	17
Функция R5J-GetSystemProperty	17

Библиотека встроенных функций Рефала-5J

В отличие от документации к Рефалу-5λ или Рефалу-05, здесь в описании функций не будет приводиться абзац о совместимости с официальной реализацией Рефала-5. Поиск подобных отличий оставлен на самостоятельное упражнение читателю.

Функции арифметики

Длинная арифметика в Рефале-5 сделана своеобразно — числа представляются как последовательности *макроцифр* — символов, хранящих число от 0 до $2^N - 1$, где N — величина, зависящая от реализации, размер макроцифры в битах. Число $s.aK \ s.aK-1 \dots s.a1 \ s.a0$ имеет значение $a_K \cdot 2^{NK} + a_{K-1} \cdot 2^{N \cdot (K-1)} + \dots + a_1 \cdot 2^N + a_0$.

Знак числа представляется необязательной литерой '+' или '-', предшествующей последовательности макроцифр. Если знак отсутствует, то подразумевается плюс.

Поскольку N обычно достаточно велико (32 или 64), то на практике числа, состоящие из нескольких макроцифр, встречаются редко, программист может считать, что число (скорее всего) помещается в символ.

Это касается и некоторых встроенных функций, функции First, Last и Lenw изображают длину выражения в термах в виде одной макроцифры, что подразумевает, что более длинные выражения реализацией не поддерживаются.

В Рефале-5J длинная арифметика сделана ещё более затейливо, но, как ни странно, более удобно для программиста. Символы-числа в ней имеют неограниченную разрядность — небольшие числа представляются объектами класса Integer, бóльшие — BigInteger, реализация встроенных функций поддерживает инвариант, что если число меньше 2^{31} , т.е. представимо положительным значением типа int, то используется Integer, иначе — BigInteger. Это надо иметь ввиду при написании собственных функций на Java.

Но, несмотря на безграничность символов-чисел, для совместимости с программами для других реализаций Рефала-5 цепочки «макроцифр» тоже поддерживаются с той же семантикой — аргумент арифметической функции, представленный как несколько символов-чисел изображает одно число как значение многочлена с коэффициентами-числами и значением переменной 2^N . Однако, параметр N здесь не привязан к реализации, а задаётся произвольно.

Размер «макроцифры» в битах задаётся системным свойством `r5jrt.macrodigitBits` и по умолчанию равным 32. Однако, пользователь может переопределить это значение, передав команде `java` опцию `-Dr5jrt.macrodigitBits=<N>`, где N — произвольное положительное число (не обязательно привычные 32 или 64).

Аргументы арифметических функций в Рефале-5J могут быть представлены цепочками макроцифр, неотрицательные числа могут начинаться на необязательный знак '+', однако, в возвращаемом значении неотрицательные числа всегда будут изображаться одним символом-числом, отрицательные — символом '-' и символом-числом.

Число, представленное цепочкой из, в общем случае, нескольких макроцифр с необязательным знаком в начале мы будем называть *длинным числом*. Длинное число будем обозначать как `e.LongNumber`:

```
e.LongNumber ::= s.Sign? s.NUMBER+
s.Sign ::= '+' | '-'
```

Функции Add, Div, Mod, Mul, Sub

Эти функции выполняют соответствующие арифметические действия: соответственно, сложение, деление, вычисление остатка от деления, умножение и вычитание. Их формат:

```
<Функция e.BinArithmArg> = '-'? s.NUMBER
```

```
e.BinArithmArg ::=
  s.Sign? s.NUMBER e.LongNumber
| (e.LongNumber) e.LongNumber
```

Аргументами, в общем случае, являются длинные числа, первый аргумент для однозначного разбора ограничивается круглыми скобками. Однако, если в первом аргументе только одна макроцифра, скобки вокруг него можно не ставить.

Для записи арифметических функций можно использовать синтаксический сахар: `<+ ...>`, `</ ...>`, `<% ...>`, `<* ...>` и `<- ...>` означают, соответственно, вызовы `Add`, `Div`, `Mod`, `Mul`, `Sub` — компилятор заменяет их на вызовы функций, названных идентификаторами. В дампе поля зрения вызов, изображённый в исходном тексте как `<+ s.X s.Y>`, будет показан как вызов функции `Add`.

В противоположность, Рефал-05 буквально поддерживает встроенные функции с именами `"+"`, `"/"` и т.д., т.е. в дампе можно будет увидеть вызовы вида `<+ ...>`.

Функция Divmod

```
<Divmod e.BinArithmArg> = ('-'? s.NUMBER) '-'? s.NUMBER
```

Функция вычисляет и частное (в скобках), и остаток. Знак остатка совпадает со знаком делимого, знак частного — '+', когда знаки делимого и делителя совпадают, и '-' в противном случае.

Функция Compare

`<Compare e.BinArithmArg> = '-' | '0' | '+'`

Функция принимает два числа в том же формате, что и другие двуместные арифметические функции и возвращает литеру, изображающую знак разности между ними.

Функция Numb

`<Numb { ' ' | '\t' }* s.Sign? s.Digit* e.ANY> = '-'? s.NUMBER`

`s.Digit ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'`

Функция преобразует десятичную запись числа в число, предварённое необязательным знаком '-'. Аргумент может начинаться на произвольное количество пробелов и знаков табуляций, за которым следует необязательный знак и последовательность десятичных цифр. За последовательностью десятичных цифр может следовать произвольное выражение, которое игнорируется.

Если аргумент не является корректной записью числа, то это не является ошибкой, функция возвращает 0.

Следует отметить, что поддерживаются только ASCII-цифры. Цифры других письменностей, также присутствующие в Юникоде, функция цифрами считать не будет.

Функция Symb

`<Symb s.Sign? s.NUMBER+> = s.Sign? s.Digit+`

Функция преобразует длинное число в его десятичную запись. Результат функции начинается на знак, только если аргумент начинался на знак, в том числе и на знак '+':
`<Symb '+' 100> = '+100'`.

Функции запроса и преобразования типов

Функция Type

`<Type e.ANY> = s.Type s.SubType e.ANY`

`s.Type s.SubType ::=`
`'*0' | 'D0' | 'L' s.Case | 'P' s.Case | 'O' s.Case`
`| 'N0'`
`| 'Wi' | 'Wq'`
`| 'B0'`
`| '00'`

`s.Case ::= 'u' | 'l'`

Функция принимает произвольное выражение и возвращает его же, предварённое типом и подтипом того, с чего начинается аргумент. Тип и подтип изображаются литерами:

- `'*0'` — аргумент пустой.
- `'D0'` — литера — ASCII-цифра. В реализации функций намеренно распознаются только ASCII-цифры, чтобы при написании лексических анализаторов можно было собрать все символы, для которых функция `Type` возвращает `D0`, и передать их в `Numb`.
- `'L'` `s.Case` — литера-буква, литера, для которой `Character.isLetter(ch)` возвращает `true`.
- `'P'` `s.Case` — печатный символ, литера для которой истинно

`Character.isDefined(ch) && ! Character.isISOControl(cp)`

- `'0'` `s.Case` — для всех остальных символов-литер.
- `'N0'` — символ-число.
- `'Wl'` — символ-слово, который можно записать в исходном тексте без кавычек, `'Wq'` — символ-слово, запись которого требует кавычек. Кавычки не требуются, если символ-слово начинается с буквы (`Character.isLetter(ch)` истинно), состоит из букв, ASCII-цифр и знаков `'-'` или `'_'`.
- `'B0'` — аргумент начинается на скобочный терм.
- `'00'` (два нуля) — любой другой объект. Откомпилированный код и встроенные функции не способны породить такие значения, однако, они могут быть порождены пользовательскими функциями, написанными на Java.

Значение `s.Case` определяет регистр — `'u'`, если верно `Character.isUpperCase(ch)` или `Character.isTitleCase(ch)`, иначе `l`. Данное значение имеет смысл только для типа `'L'`, для типов `'P'` и `'0'` подтип смысла не несёт, но добавлен по аналогии с Рефалом-5.

Функция **Implode**

```
<Implode e.Indent e.ANY> = s.WORD e.ANY  
<Implode e.ANY> = 0 e.ANY
```

```
e.Ident ::= s.Letter { s.Letter | s.Digit | '-' | '_' | '$' }*  
s.Letter ::= ... -- Character.isLetter истинно
```

Если аргумент начинается с записи идентификатора, то функция возвращает соответствующий идентификатор и остаток аргумента. Иначе, функция возвращает аргумент как есть, предваряя его макроцифрой 0.

Идентификатор должен начинаться на букву, за которой следует ноль или более букв, цифр и знаков '-', '_', '\$'. Знак доллара здесь — недокументированное расширение Рефала-5, по аналогии поддерживается и Рефалом-5J.

Функция **Implode_Ext**

```
<Implode_Ext s.CHAR*> = s.WORD
```

Аргумент должен состоять из нуля и более символов-литер, функция возвращает символ-слово с соответствующим текстовым представлением. Если аргумент пустой, возвращается пустой символ-слово "".

Функции **Explode** и **Explode_Ext**

```
<Explode s.WORD> = s.CHAR*  
<Explode_Ext s.WORD> = s.CHAR*
```

Две функции с идентичой семантикой — принимают символ-слово и возвращают последовательность образующих его литер.

Функции **Chr**, **Lower**, **Ord**, **Upper**

```
<Chr e.ANY> = e.ANY  
<Lower e.ANY> = e.ANY  
<Ord e.ANY> = e.ANY  
<Upper e.ANY> = e.ANY
```

Функции проходят по выражению, в том числе и внутри круглых скобок, и соответствующим образом трансформируют символы:

- Chr превращает все символы-числа в литеры с кодом Юникода, полученным делением числа на $110000_{16} = 1114112_{10}$ — максимальное количество кодовых точек Юникода.

- Lower превращает символы-литеры в эквивалентные литеры нижнего регистра, для трансформации используется метод `int Character.toLowerCase(int)`.
- Ord превращает все символы-литеры в символы-числа с соответствующим кодом Юникода.
- Upper превращает символы-литеры в эквивалентные литеры верхнего регистра, используя `int Character.toUpperCase(int)`.

Функции работы со стеками

Функция Br

`<Br e.Key '=' e.Value> = пусто`

`e.Key ::= { t.ANY \ '=' }*`
`e.Value ::= e.ANY`

Функция закапывания (bury).

Сохраняет значение `e.Value` на вершине стека по имени `e.Key`, если такого стека не существовало, он создаётся. Аргумент на верхнем уровне должен содержать знак равенства

Функция Cp

`<Cp e.Key> = e.Value`

Функция копирования (copy).

Возвращает копию вершины стека `e.Key`, сам стек не меняется. Если стек не существовал, возвращается пустота.

По аналогии с Рефалом-5, содержит ошибку. Код

`<Br 'A=B=C'> <Cp 'A=B'>`

вернёт 'C'.

Функция Dg

`<Dg e.Key> = e.Value`

Функция выкапывания (digg).

Возвращает значение с вершины стека `e.Key`, удаляя его оттуда. Если стек не существовал, возвращается пустота.

По аналогии с Рефалом-5, содержит ошибку. Код

`<Br 'A=B=C'> <Dg 'A=B'>`

вернёт 'C'.

Функция Rp

`<Rp e.Key '=' e.Value> = пусто`

Функция замены (replace).

Заменяет содержимое вершины стека `e.Key` на значение `e.Value`, старое значение при этом теряется. Если стек не существовал, создаётся новый, на вершину которого кладётся значение `e.Value`.

Функции ввода-вывода

Консольный ввод-вывод, функции Card, Print, Prout

`<Card> = s.CHAR* 0?`

`<Print e.ANY> = e.ANY`

`<Prout e.ANY> = пусто`

Функция Card считывает ~~перфокарту~~ очередную строку со стандартного ввода, если был обнаружен конец файла, то ввод завершается макроцифрой 0.

Функция Print печатает произвольное выражение на стандартный вывод, возвращает свой аргумент, функция Prout (print out) возвращает пустоту. Обе функции после вывода выражения осуществляют переход на новую строку.

Обе функции печатают литеры как есть, круглые скобки — как (и), символы-числа в десятичной форме, символы-слова — как текст. После символов-чисел и символов-слов принудительно печатается пробел, чтобы иметь возможность напечатать несколько чисел или слов подряд без принудительной вставки пробелов между ними. Если же пробел нежелателен, то их нужно преобразовать в цепочки литер, используя Symb или Explode соответственно.

Текстовое представление выражения не взаимно-однозначно — при печати невозможно отличить круглые скобки от литер-скобок, число 100500 от символа-слова "100500" или последовательности литер '100500 ' (пробел в конце) и т.д.

Если аргумент содержит символ неизвестного типа, то распечатается его текстовое представление, возвращённое методом `.toString()`.

Консольный ввод-вывод по умолчанию выполняется в кодировке UTF-8, можно указать другую кодировку в системном свойстве `r5jrt.consoleEncoding`. Например, для консоли Windows имеет смысл добавить опцию `-Dr5jrt.consoleEncoding=CP866` к интерпретатору Java.

Дескрипторы файлов, функции Open и Close

Файловый ввод-вывод в Рефале-5 и, соответственно, в Рефале-5J устроен архаично. Открытым файлам пользователь сопоставляет номера дескрипторов, глобальные на всю программу.

Для открытия файла и связывания его с дескриптором используется функция

`<Open s.Mode s.Handle e.FileName?> = пусто`

`s.Mode ::= 'r' | 'w' | 'a' | r | w | a | rb | wb | ab`

`s.Handle ::= s.NUMBER`

`e.FileName ::= s.CHAR+`

Режим доступа определяется параметром `s.Mode`:

- параметры 'r', 'w', 'a', r, w, a определяют текстовый режим работы, параметры rb, wb, ab — двоичный режим,
- 'r', r, rb — открытие текстового файла для чтения, файл должен существовать;
- 'w', w, wb — открытие текстового файла для (пере)записи, если файл не существовал, он создаётся, если существовал — перезаписывается;
- 'a', a, ab — открытие текстового файла для дозаписи, если файл не существовал, он создаётся, если существовал — новые данные дописываются в конец.

Разницы между 'r', 'w' и 'a' с одной стороны и r, w и a с другой нет, исторически в Рефале-5 изначально режим задавался литерой, затем добавили символ-слово, который как есть передавался в функцию `fopen()` языка Си.

Номер файла задаётся параметром `s.Handle`. Максимальное число открытых файлов ограничено параметром `r5jrt.fileLimit`, номер реального дескриптора определяется как остаток от деления на `r5jrt.fileLimit`. Минимальное допустимое значение этого параметра — 20, как это упоминается в учебнике Турчина (см. раздел 2.3 Input-Output). Значение по умолчанию — 40, как в актуальной реализации Рефала-5 Version ПЗ.

В текстовом режиме принудительно используется кодировка UTF-8 и это поведение в актуальной версии компилятора никак нельзя изменить. В двоичном режиме каждый выводимый символ преобразуется в байт путём выбора младших 8 бит кодовой точки, для вводимых символов выполняется обратное преобразование. Т.е. двоичный ввод можно считать эквивалентным вводу в кодировке Latin-1.

Второе отличие текстового и двоичного режимов — в текстовом режиме при чтении маркерами перевода строки считаются последовательности литер '\n' и '\r\n', в двоичном — только '\n'. При записи тестовых файлов разделителем строчек является платформенно-зависимый разделитель, двоичных файлов — '\n'.

Имя файла может отсутствовать, в этом случае будет открыт файл с именем по умолчанию `REFAL<номер>.DAT`. Например, `<Open w 10>` откроет файл `REFAL10.DAT` в режиме перезаписи.

Функцию `Open` можно применять к ранее открытому дескриптору файла, в этом случае ранее открытый дескриптор будет закрыт, а новый файл открыт.

Функция `Close` закрывает ранее открытый файл:

`<Close s.Handle> = пусто`

Как и для функции `Open`, реальный номер дескриптора вычисляется как остаток от деления на системный параметр `r5jrt.fileLimit`. Если соответствующий дескриптор не был открыт, функция `Close` ничего не делает.

Чтение и запись файлов, функции `Get`, `Put`, `Putout`, `Write`

```
<Get s.Handle> = s.CHAR* 0?  
<Put s.Handle e.ANY> = e.ANY  
<Putout s.Handle e.ANY> = пусто  
<Write s.Handle e.ANY> = пусто
```

Дескриптор файла определяется как остаток от деления значения `s.Handle` на системный параметр `r5jrt.fileLimit`. Дескриптор `0` при чтении соответствует стандартному вводу, при записи — стандартному потоку вывода ошибок (`stderr`).

Функция `Get` считывает очередную строку из файла. Маркер перевода строки к считанным данным не добавляется. Если при чтении встретился конец файла, в конец возвращаемого значения добавляется символ-число `0`.

Вызовы `<Card>` и `<Get 0>` идентичны.

Функции `Put` и `Putout` печатают строку в файл, завершая её маркером перевода строки. Функция `Put` возвращает выражение, подлежащее печати, функция `Putout` возвращает пустоту. Функция `Write` аналогична `Putout`, только маркер перевода строки не добавляет.

Функции `Put`, `Putout` и `Write` преобразуют выражение в текст таким же образом, как и функции `Print` и `Prout`.

Заметим, что для `Write` не предусмотрено парной функции, осуществляющей вывод на стандартный поток вывода без добавления перевода строки. Вывести текст без последующего перевода строки можно только в файл или `stderr`.

Операции с неоткрытым дескриптором или дескриптором, открытым не в том режиме, ошибкой не являются. Если дескриптор ненулевой и не был открыт для чтения, то перед первой операцией `<Get s.Handle>` неявно будет вызвана `<Open r s.Handle>`, аналогично для операций записи файл неявно открывается как `<Open w s.Handle>`. Следствие — неявно открываемые файлы будут иметь имя по умолчанию `RE-FAL<номер>.DAT`.

Заметим, что в Рефале-5 присутствует баг: можно открыть файл номер `0`, однако прочитать из или записать в него что-либо будет невозможно. Операции чтения с нулевым дескриптором будут читать стандартный ввод, операции записи — писать на стандартный вывод ошибок. Рефал-5J воспроизводит этот баг.

Функция `ExistFile`

```
<ExistFile e.FileName> = True | False
```

Функция проверяет существование файла при помощи `java.nio.file.Files.exists`.

Функция `RemoveFile`

```
<RemoveFile e.FileName>  
= True ()
```

```
= False (e.ErrorMessage)
```

```
e.ErrorMessage ::= s.CHAR+
```

Функция пытается удалить файл, если не удалось, возвращается сообщение об ошибке. Для удаления используется функция `java.nio.file.Files.delete`, поэтому при помощи `RemoveFile` можно удалить как файл, так и пустую папку.

Взаимодействие с операционной системой

Функция Arg

```
<Arg s.NUMBER e.Default> = s.CHAR* | e.Default
```

```
e.Default ::= e.ANY
```

Функция возвращает значение указанного параметра командной строки. После номера параметра указывается значение по умолчанию, которое будет возвращено, если номер параметра больше их фактического количества в командной строке. Если значение по умолчанию не указывать, то, очевидно, значением по умолчанию будет пустое выражение — в этом случае семантика функции `Arg` в Рефале-5J будет совпадать с семантиками других реализаций Рефала-5.

В метод `main` виртуальная машина Java передаёт только аргументы после имени программы, т.е. параметры, начиная с `<Arg 1>`. Для реконструкции значения `<Arg 0>` используется недокументированное системное свойство `sun.java.command`. Если попадётся реализация Java, где оно не установлено, то `<Arg 0>` вернёт пустое выражение.

Функция GetEnv

```
<GetEnv s.CHAR*> = s.CHAR*
```

Функция возвращает значение соответствующей переменной среды. Если переменная среды с заданным именем не найдена, возвращается пустое выражение. Для пустого аргумента также возвращается пустое выражение.

Функция Exit

```
<Exit {'+' | '-'}? s.NUMBER> = нет возврата
```

Функция завершает работу программы с соответствующим кодом возврата. Фактический код возврата зависит от операционной системы, в частности, на Linux учитываются только младшие 8 бит — если программа завершится как `<Exit '-' 3>`, операционная система увидит код возврата 253.

Функция System

`<System e.Command> = '-'? s.NUMBER`

Функция запускает команду в оболочке операционной системы и возвращает её код возврата. В случае каких-то проблем с запуском функция может вернуть код '-' 1.

Функция Time

`<Time> = s.CHAR+`

`<Time ISO> = s.CHAR+`

Функция возвращает строку с датой и временем. При вызове без параметров время возвращается в формате функции `ctime()` языка Си, но без перевода строки в конце: `Www Mmm dd hh:mm:ss уууу`, например `'Mon Mar 09 02:51:27 2026'`. Вызов `<Time ISO>` выводит время в формате ИСО: `уууу-mm-ddThh:mm:ss.ssssss`, например, `'2026-03-09T02:51:27.831705100'`.

Функции работы с выражениями

Функция Lenw

`<Lenw e.ANY> = s.NUMBER e.ANY`

Функция возвращает свой аргумент, предвеля его длиной аргумента в термах.

Функции First и Last

`<First s.Len e.Expr> = (e.Prefix) e.Suffix`

`<Last s.Len e.Expr> = (e.Prefix) e.Suffix`

`s.Len ::= s.NUMBER`

`e.Expr, e.Prefix, e.Suffix ::= e.ANY`

`e.Expr : e.Prefix e.Suffix`

Функция `First` отделяет от выражения максимально длинный префикс не более чем указанной длины, функция `Last` отделяет от выражения максимально длинный суффикс не более чем указанной длины.

Запросы к среде исполнения

Функция ListOfBuiltin

`<ListOfBuiltin> ::= (s.FuncNo s.FuncName s.FuncType)+`

```
s.FuncNo ::= s.NUMBER
s.FuncName ::= s.WORD
s.FuncType ::= regular | special
```

Функция возвращает таблицу встроенных функций. Параметр `s.FuncNo` используется внутри официальной реализации Рефала-5, функция `ListOfBuiltin` в Рефале-5J возвращает те же значения, что и официальная реализация.

Параметр `s.FuncType` возвращает `regular` для обычных функций, `special` для метафункций.

Актуальная `ListOfBuiltin` в Рефале-5J возвращает только те регулярные функции, которые реализованы. По техническим причинам в Рефале-05 эта функция должна возвращать все метафункции, в том числе и нереализованные, для совместимости с Рефалом-05 в Рефале-5J тоже перечисляются все метафункции.

Функция Step

```
<Step> = s.NUMBER
```

Функция возвращает количество выполненных шагов рефал-машины. Сопоставление с образцом в условии и вызов блока также считаются как один шаг.

Функция TimeElapsed

```
<TimeElapsed 0?> = s.CHAR+
```

Функция возвращает время, прошедшее с последнего запуска `<TimeElapsed 0>` или, если функция `TimeElapsed` до этого ни разу не вызывалась с нулём, с начала выполнения программы.

Функция возвращает прошедшее время в секундах в виде дробного десятичного числа вида 123.456.

Функция SizeOf

```
<SizeOf s.CharType> = s.NUMBER
```

```
s.CharType ::= 'c' | 's' | 'i' | 'l' | 'p'
```

Функция возвращает размер примитивного типа данных языка Си, на котором написан интерпретатор `refgo`, используется в компиляторе `srefal` для резервирования соответствующего количества байт в генерируемом коде на языке сборки.

Параметр `'c'`, `'s'`, `'i'`, `'l'`, `'p'` означает, соответственно, размер `char` (всегда 1), `short`, `int`, `long` и указателя. Поскольку актуальный компилятор реализован для платформы Java, эти возвращаемые значения для `'s'`, `'i'`, `'l'` и `'p'` сделаны настраиваемыми и имеют значения по умолчанию `Short.BYTES`, `Integer.BYTES`,

Long.BYTES и Long.BYTES, соответственно. Значение по умолчанию можно переопределить, задав системное свойство `r5jrt.sizeOf.<буква>`, например, `-Dr5jrt.sizeOf.p=4, -Dr5jrt.sizeOf.i=2`.

Возвращаемое значение для `<SizeOf 'c'>` всегда 1 и его переопределить невозможно.

Метафункции

В актуальной версии реализованы только `Mu` и `Residue`, хотя функция `ListOfBuiltin` возвращает имена всех четырёх (ещё и `Up` и `Ev-met`) — см. `ListOfBuiltin`. Вызовы последних двух в программе не будет синтаксической ошибкой, однако при их выполнении программа завершится с исключением `AssertionError` с сообщением «Not implemented».

Метафункции используются для вызова других функций по их имени. Поиск осуществляется в два этапа:

1. Поиск среди всех функций (`entry`- и локальных) того файла, где записан вызов метафункции.
2. Поиск среди всех `entry`-функций программы и встроенных функций.

Функции `Mu` и `Residue`

```
<Mu t.Callee e.ANY> = e.ANY  
<Residue t.Callee e.ANY> = e.ANY
```

```
t.Callee ::= s.WORD | (s.CHAR*) | s.CHAR
```

Функции `Mu` и `Residue` принимают имя и аргумент функции и вызывают функцию с указанным аргументом.

Для вызова функции `Residue` можно использовать синтаксический сахар `<? ...>`, однако, в отличие от Рефала-05, функции с именем «?» не существует, подобные вызовы будут оттранслированы в вызовы `Residue` — подробнее см. комментарий к функции `Add`.

Имя функции может быть задано или символом-словом, или последовательностью литер в круглых скобках, или в виде одной литеры. В качестве имени функции можно использовать и символы `'%'`, `'*'`, `'+'`, `'-'`, `'/'`, `'?'`, означающие функции с именами соответствующего синтаксического сахара.

В Рефале-5 разница в семантике между `Mu` и `Residue` проявляется при вычислениях под метакодом, в обычном режиме они синонимы. В актуальной версии Рефала-5J функция для вычисления под метакодом `Ev-met` не реализована, поэтому здесь `Mu` и `Residue` точные синонимы.

Генерация случайных чисел

Функция RandomDigit

```
<RandomDigit '+'? s.NUMBER> = s.NUMBER
```

Функция возвращает случайное число от нуля до своего аргумента включительно. Знак '+' в аргументе ни на что не влияет. Поскольку размер символов-чисел в Рефале-5J не ограничен, можно породить сколь угодно большие случайные числа.

Функция Random

```
<Random '+'? s.NUMBER> = s.NUMBER+
```

Функция возвращает непустую последовательность макроцифр длиной не более, чем аргумент функции. Знак '+' ни на что не влияет. Каждая из возвращённых макроцифр не больше, чем $2^{r5jrt.macrodigitBits}$, где в показателе степени находится значение системного свойства `r5jrt.macrodigitBits`.

Системные функции, сериализация выражений

Функция `Sysfun` изначально была недокументированной и предназначенной для недокументированных расширений интерпретатора. В частности, она использовалась для ускорения десериализации в функции `Xxin` библиотеки `reflib`. Формат функции `Sysfun` предполагался следующий

```
<Sysfun s.FuncNo e.Arg> = e.Res
```

где `s.FuncNo` — номер недокументированной функции, аргумент и результат зависят от этого номера. Но поддерживалась только функция № 1 — десериализация.

В дальнейшем по недоразумению была добавлена функция `DeSysfun`, выполняющая сериализацию. В документации она заявлена как обратная к `Sysfun`: «The function is inverse to Sysfun» — из файла `news.txt` дистрибутива. В исходниках также встречается такой комментарий:

```
/* <Sysfun 1 e.inp-file-name>
   It is unclear the meaning of the first argument.
   It seems it is just a rudiment ... Nemytykh, 28-03-2023.
*/
```

Чтобы как-то навести порядок, операция сериализации для симметрии сделана системной функцией № 2, а функция `DeSysfun` выдаёт предупреждение о том, что она не рекомендована (`deprecated`).

Функция Sysfun

```
<Sysfun 1 e.InputFileName> = e.ANY  
<Sysfun 2 e.OutputFileName (s.Width e.ANY)> = пусто
```

```
s.Width ::= s.NUMBER  
e.InputFileName, e.OutputFileName ::= s.CHAR+
```

Системная функция № 1 выполняет десериализацию объектного выражения из файла с указанным именем, системная функция № 2 выполняет сериализацию произвольного объектного выражения. Параметр `s.Width` определяет длину строк в сериализованном представлении.

Формат сериализованных данных напоминает формат литералов Рефала-5, но есть и отличия. Правила сериализации:

- В сериализованной форме допустимы только символы ASCII.
- Пробелы и табуляции вне кавычек служат для разделения образов идентификаторов и чисел. При сериализации после вывода каждого незакавыченного идентификатора и каждого числа принудительно после него добавляется пробел. При десериализации пробелы и табуляции игнорируются.
- Переводы строк (представленные как `\n`, `\r` или `\r\n`) игнорируются везде: они могут рассекать запись идентификатора, числа, escape-последовательности и т.д. При сериализации переводы строк вставляются после вывода каждых `s.Width` символов.
- Несколько смежных литер, как и в исходном коде на Рефале, могут быть записаны под общими одинарными кавычками (апострофами). При сериализации литеры всегда записываются в кавычках.
- Символы-числа записываются в десятичной форме, и, как сказано выше, после них всегда следует знак пробела.
- Составной символ считается записанным в идентификаторной форме, если он начинается с ASCII-буквы, состоит из ASCII-букв, ASCII-цифр и знаков минуса и прочерка. Составной символ в идентификаторной форме может быть записан без кавычек, при сериализации такие символы всегда сохраняются без кавычек, после них добавляется один пробел.
- Остальные составные символы пишутся в двойных кавычках, при сериализации после закрывающей кавычки пробел не добавляется.
- Внутри как одинарных, так и двойных кавычек можно использовать escape-последовательности:

- \n, \r, \t — перевод строки, возврат каретки, табуляция,
- \', \", \\ — одинарная, двойная кавычки, обратная дробь,
- \(\, \), \<, \> — знаки круглых и угловых скобок, при сериализации эти знаки всегда пишутся как escape-последовательности, хотя формально они избыточны,
- \xHH, \uNNNNNN — символы, заданные шестнадцатеричным кодом, \xHH позволяет указывать коды до 255, \uNNNNNN — любой допустимый код Юникода.

При сериализации используется следующий порядок:

- если символ можно изобразить «односимвольной» escape-последовательностью, используется она (в том числе и для скобок),
 - если символ является печатным ASCII-символом, то он выводится как есть,
 - если код символа меньше 255, он выводится как \xHH,
 - иначе используется запись \uNNNNNN.
- Скобочные термины записываются в круглых скобках.
 - При десериализации литеры также распознаются и вне апострофов, в том числе и заданные escape-последовательностями.

Функция DeSysfun

<DeSysfun e.OutputFileName (s.Width e.ANY)> = пусто

Выводит предупреждение на стандартный поток ошибок о том, что она не рекомендована и выполняет сериализацию как системная функция № 2. См. документацию к <Sysfun 2 ...>.

Собственные функции данной реализации

Функция R5J-GetSystemProperty

\$EXTERN R5J-GetSystemProperty;

<R5J-GetSystemProperty s.CHAR+> = s.CHAR* | null

Функция возвращает системное свойство — является обёрткой над методом String System.getProperty(String) языка Java. Если соответствующее системное свойство не задано, возвращается символ null.

Из соображений совместимости, функция отсутствует в списке ListOfBuiltin, поэтому для использования в программе её нужно подключить при помощи \$EXTERN.